گزارش فنی — تکمیل و سخت‌سازی سرویس Banking Identity

ممیزی، بازطراحی و پیاده‌سازی سرویس Identity برای سامانهٔ بانکی مبتنی بر Microservice

تاریخ: 19-10-2026 ریپازیتوری: `banking-identity` شاخه: `main` (تغییرات uncommitted)

۱. خلاصه اجرایی

ریپازیتوری از یک اسکلت ناقص (یک Entity، یک Use Case نیمه‌کاره، سه Port، بدون Persistence و API و Migration) به یک سرویس Identity کامل و تست‌شده تبدیل شد. معماری نهایی، Clean Architecture عملیاتی با یک جهت وابستگی `presentation → application → domain ← infrastructure` است که فقط در یک نقطهٔ ترکیب (`create_app()`) سیم‌کشی می‌شود. تنها دپندسی افزوده‌شده `argon2-cffi==25.1.0` است.

سنجه	مقدار
تست‌های پاس‌شده	161 (144 unit + 17 integration on real PostgreSQL 17)
پوشش کد	92% (1235 statements, 97 miss)
Type checking	mypy clean — 60 files, disallow_untyped_defs
Lint / Format	ruff check + ruff format --check — pass
Migration	9ce5b1490bff — upgrade/downgrade verified on live DB
Smoke زنده	register → verify → login → logout → reuse-401 با uvicorn و PostgreSQL واقعی
دپندسی جدید	argon2-cffi==25.1.0

۲. یافته‌های ممیزی اولیه

#	یافته	شدت	اصلاح
A1	اسکریپت `banking-identity = "banking_identity:main"` به ماژول ناموجود اشاره می‌کرد؛ build شکست می‌خورد	Blocker	`module-name = "identity_service"` + `identity_service.main:run`
A2	دامنه با `ValueError` کار می‌کرد — نقض Domain Exception Hierarchy	High	سلسله‌مراتب `DomainError` با ۱۲ exception اختصاصی

A3	پورت `VerificationSender` توکن خام را نمی‌گرفت — فلوی verification غیرقابل پیاده‌سازی	High	امضای جدید `(to_email, raw_token, expires_at)`
A4	چک `exists_by_email` بدون پشتیبانی constraint — ریسک race در ثبت‌نام هم‌زمان	High	پیش‌چک + `UNIQUE(email)` + ترجمهٔ `IntegrityError`
A5	نبود policy رمز عبور، نرمال‌سازی کامل ایمیل، issuance توکن، session و logout	High	پیاده‌سازی کامل — بخش ۴
A6	`Dockerfile`، `.env.example`، `src/main.py` خالی؛ بدون `alembic/`	Medium	تکمیل همه
A7	`ruff format --check` روی ۹ فایل شکست می‌خورد	Low	فرمت یکدست

۳. تصمیم‌های معماری (ADR)

ADR-001 — توکن Session مات (Opaque) به‌جای JWT

الزام کسب‌وکار، ابطال فوری است (logout، تغییر/ریست رمز، deactivate). JWT بدون denylist ابطال تک‌توکن ندارد و version-claim فقط ابطال دسته‌ای پس از تغییر credential می‌دهد. تصمیم: توکن تصادفی ۲۵۶ بیتی، ذخیرهٔ فقط SHA-256، resolve هر request از دیتابیس (lookup ایندکس‌شده). بها: یک query ایندکس‌دار در هر request درخواست‌محور.

ADR-002 — Argon2id با rehash-on-login

`argon2-cffi` با پارامترهای پیش‌فرض کتابخانه (argon2id، m=64MiB، t=3، p=4) مطابق RFC 9106. متد `needs_rehash` در پورت، ارتقای شفاف پارامترها را در login بعدی ممکن می‌کند.

ADR-003 — SHA-256 برای توکن‌ها، KDF برای رمز عبور

رمز آنتروپی کم دارد و KDF کند می‌خواهد؛ توکن ۲۵۶ بیت آنتروپی دارد، پس SHA-256 سریع کافی و صحیح است: سرقت دیتابیس ≠ امکان replay، ولی lookup ارزان می‌ماند. این تفکیک اصل طراحی است، نه سلیقه.

ADR-004 — مصرف اتمیک توکن با UPDATE شرطی

زیر `READ COMMITTED`، تراکنش دوم روی row-lock می‌ایستد، پس از commit اولی predicate را دوباره ارزیابی می‌کند، صفر سطر می‌گیرد و `None` → خطای یکسان `InvalidTokenError`. اثبات تجربی: تست ۴-thread با دقیقاً یک برنده.

```
UPDATE email_verification_tokens
SET used_at = :now
WHERE token_hash = :h AND used_at IS NULL AND expires_at > :now
RETURNING *;
```

ADR-005 — Session-per-request به‌عنوان Unit of Work

Repositoryها فقط `flush` می‌کنند (بروزرسانی زودهنگام نقض constraint)؛ commit/rollback در مرز request. پیامد: اگر consume توکن موفق ولی مرحله بعدی شکست بخورد، rollback کل تراکنش توکن را «نمی‌سوزاند».

ADR-006 — ارسال ایمیل هرگز بخشی از تراکنش اتمیک نیست

ارسال پس از persist انجام و شکستش فقط log می‌شود (`EmailDeliveryError` → `email.delivery_failed`). برای reset-request، بازگرداندن خطا به‌خاطر شکط ایمیل خودش نشانه‌ی وجود حساب بود. ارتقا: Transactional Outbox — مستند، خارج از scope فعلی.

ADR-007 — Policy نرمال‌سازی ایمیل

lowercase + trim کامل؛ حفظ plus-tag و نقطه‌ها؛ همان policy در schema و use case و lookup دیتابیس؛ `UNIQUE(email)` ضامن نهایی در برابر race.

ADR-008 — Rate limiting درون‌پردازشی پشت Port

پنجرهٔ لغزان per-IP روی چهار scope حساس (register, login, resend, reset-request). محدودیت per-instance بودن صریحاً مستند است؛ قرارداد `RateLimiter` ثابت می‌ماند و impl ردیس‌بی بعداً جایگزین می‌شود.

ADR-009 — Reactivation با fail-closed

کلید سرویس‌به‌سرویس از env (`X-Admin-Key`)، مقایسه در زمان ثابت (`secrets.compare_digest`)، و در نبود کلید، endpoint برای همه 403 است — نه fallback باز.

ADR-010 — توکن‌ها «رکورد دامنهٔ سبک»اند، نه Entity کامل

تنها invariant آن‌ها تک‌مصرفی و انقضاست: در دامنه با `OneTimeToken.consume()` و در دیتابیس با UPDATE اتمیک اعمال می‌شود. Entity کامل با identity-continuity برای آن‌ها abstraction بی‌مصرف بود.

۴. جزئیات پیاده‌سازی لایه‌به‌لایه

۴.۱ Domain — ماشین حالت و invariantها

`Identity` یک dataclass خالص بدون هیچ import زیرساختی است:

```
PENDING_VERIFICATION ──verify_email──> ACTIVE ──deactivate──> DEACTIVATED
        │                                                        │
        └──────────────────── deactivate ────────────────────────┘
       reactivate: ACTIVE اگر email_verified_at موجود، وگرنه PENDING_VERIFICATION
```

- `verify_email`: رد روی DEACTIVATED با `IdentityNotActiveError` — verification هرگز deactivate را implicitly فعال نمی‌کند؛ رد روی تکرار با `EmailAlreadyVerifiedError`.
- `change_password` فقط برای ACTIVE؛ `apply_password_reset` بدون تغییر state — حساب deactivated پس از reset همچنان غیرقابل احراز هویت می‌ماند.
- `customer_id`: فیلد immutable بدون mutator — ثبات مرجع به Customer Service در سطح type.

۴.۲ Application — قرارداد پورت‌ها

Semantics بازگشت/خطا صریح مستند شده: lookupها `None` برمی‌گردانند (نه exception)؛ `add` در نقض uniqueness exception دامنه‌ای raise می‌کند؛ `consume_by_token_hash` قرارداد «دقیقاً یک‌بار در کل عمر توکن» دارد؛ `revoke` idempotent است. Use caseها ورودی/خروجی dataclass صریح دارند (`LoginResult`، `AuthContext`)، هرگز آبجکت HTTP برنمی‌گردانند و TTLها به‌صورت اسکالر از composition root تزریق می‌شوند.

۴.۳ Persistence — schema و ایندکس‌ها

جدول	Constraint / Index
identities	UNIQUE(email), UNIQUE(customer_id), CHECK(status IN (...)), timestamptz
email_verification_tokens / password_reset_tokens	UNIQUE(token_hash: CHAR(64)), FK ON DELETE CASCADE, INDEX(identity_id)
auth_sessions	UNIQUE(token_hash), FK CASCADE, INDEX(identity_id)

- ایندکس روی `identity_id` صریحاً اضافه شد، چون PostgreSQL روی FK ایندکس نمی‌گذارد و `invalidate_all_for_identity` / `revoke_all_for_identity` روی آن فیلتر می‌کنند.
- ترجمهٔ خطا با `IntegrityError.orig.diag.constraint_name` → تفکیک `IdentityAlreadyExistsError` از `CustomerAlreadyLinkedError` در سطح 409.

۴.۴ Presentation

- Error envelope یکسان: `{"error": {code, message, request_id, details?}}`
- نگاشت Domain→HTTP: خطاهای اعتبارسنجی 422، تعارض‌های وضعیت 409، `InvalidTokenError` 400، `AuthenticationError` 401 (+ `WWW-Authenticate`)، `InvalidCurrentPasswordError` 403
- هندلر `RequestValidationError` عمداً `loc/msg/type` برمی‌گرداند و `input/ctx` را حذف می‌کند — pydantic مقدار خام (ازجمله رمز) را در خطا embed می‌کند (CWE-209)

- Middleware به‌صورت pure ASGI (نه `BaseHTTPMiddleware`): تولید/پذیرش `X-Request-ID`، تزریق به `ContextVar` با propagate به threadpool سینک، لاگ دسترسی با route template به‌جای path خام
- جداسازی liveness (`/health`) از readiness (`/health/ready` با `SELECT 1`)

۵. مهندسی امنیت — پوشش تهدید

تهدید	کنترل	اثبات
Account enumeration از API	پاسخ یکسان 202 برای resend/reset-request؛ 401 یکسان برای همه حالات login	تست byte-to-byte بدنه‌ها (با حذف request_id)
Timing side-channel روی وجود ایمیل	`verify_dummy` با هش Argon2 ثابت برای principal ناشناس	تست: `dummy_verifications == 1`
Replay توکن	مصرف اتمیک تک‌بار	تست reuse + تست thread-۴
Race ثبت‌نام هم‌زمان	constraint + ترجمه خطا → 409	تست integration با دو session
نشت از دیتابیس	فقط hash (Argon2id / SHA-256)، بدون raw token و plaintext	بازبینی schema + تست mapping
نشت در لاگ	allowlist فیلدها؛ فقط کلاس exception بدون traceback خام؛ sanitized 422	اسکن لاگ smoke test: صفر مورد
Brute force	429 + `Retry-After` روی چهار scope	تست API با limit=2

ماتریس چرخه credential: logout → فقط session جاری؛ تغییر رمز → همهٔ sessionها + بی‌اعتبارسازی reset tokenهای باز؛ reset → همهٔ sessionها؛ deactivate → همهٔ sessionها + هر دو نوع توکن.

۶. راهبرد تست

- **Unit (۱۴۴):** همهٔ پورت‌ها با fake درون‌حافظه‌ای؛ `FakeClock` برای سناریوهای انقضا بدون `sleep`؛ Harness که همان wiring پراکشن را بازتولید می‌کند تا تست API و تست use case هم‌ارز باشند.
- **API:** قرارداد HTTP با `dependency_overrides` و بدون دیتابیس؛ شامل تست 429، envelope خطا، و «عدم اکو شدن رمز در خطای اعتبارسنجی».
- **Integration (۱۷):** هر تست چرخهٔ واقعی `downgrade base → upgrade head` را اجرا می‌کند (تأیید دائمی migration + ایزولاسیون)؛ شامل race مصرف توکن چندنخی، CHECK constraint، و CASCADE.
- **Smoke زنده و Docker:** چرخهٔ کامل روی uvicorn و PostgreSQL واقعی؛ build دو مرحله‌ای و اجرای کانتینری `/health`.
- **Skip ایمن:** اگر `TEST_DATABASE_URL` در دسترس نباشد، integration کل ماژول skip می‌کند، نه fail.

۷. ماتریس تأیید

Command	نتیجهٔ مشاهده‌شده
uv sync / uv add argon2-cffi	argon2-cffi 25.1.0 نصب شد؛ خطای build قبلی رفع شد
alembic upgrade head	مهاجرت 9ce5b1490bff اعمال؛ schema (جدول‌ها، FK، CHECK، unique، ایندکس) تأیید شد
pytest tests/unit -v	passed 144
pytest	passed (144 + 17 integration) 161
pytest --cov=identity_service	TOTAL 92% — 1235 stmts / 97 miss
mypy src	Success: no issues found in 60 source files
ruff check / format --check	All checks passed / 82 files already formatted
uvicorn smoke (JSON logs)	چرخهٔ کامل HTTP درست؛ request_id منتشر شد؛ اسکن منفی رمز/توکن در لاگ
docker build + run	ایمیج build شد؛ /health و /docs پاسخ 200

۸. بدهی فنی و ریسک‌ریجیستر

ریسک	شدت	طرح کاهشی
Rate limiter فقط per-IP / per-instance	High برای production	impl ردیس‌بی پشت همان Port
بدون Outbox — ایمیل best-effort	Medium	Transactional Outbox (گام بعدی)
پشت proxy مشترک، کلید IP بهینه نیست	Medium	trusted-proxy handling / کلید ترکیبی IP+account
فاقد breach-check، lockout، 2FA رمز	Medium	نسخهٔ v2
فاقد exporter متریک / tracing	Low	ورودی JSON برای Loki/Grafana از قبل مهیاست

اقدام محیطی: در کانتینر dev شما، role و database تست (`identity_test` / `banking_identity_test`) به‌صورت صرفاً additive ساخته شد. حذف: `DROP DATABASE banking_identity_test; DROP ROLE identity_test;`

۹. گام بعدی توصیه‌شده

1. ساخت **Customer Service** با ارجاع به `identity.id` (یا ارسال `customer_id` هنگام ثبت‌نام — مرجع پایدار و immutable از قبل در جای خود است).
2. افزودن Transactional Outbox برای تحویل تضمینی ایمیل.
3. Rate limiter ردیس‌بی + محدودیت per-account (نه فقط per-IP).

4. Endpoint بازرسی session برای سرویس‌های دیگر (احراز هویت در gateway).

تغییرات در working tree ریپازیتوری `banking-identity` قرار دارد و هنوز commit نشده است. این گزارش در تاریخ 2026-10-19 از وضعیت واقعی کد و نتایج اجرای مستقیم دستورات تولید شده است؛ هیچ ادعایی بدون اجرای مشاهده‌شده در آن ثبت نشده است.